Problem 01

```
bestProfit = -MAX_INT
bestBuyDate = None
bestSellDate = None
for i = 1 to n:
    for j = i + 1 to n:
        if price[j] - price[i] > bestProfit:
            bestBuyDate = i
            bestSellDate = j
            bestProfit = price[j] - price[i]
return (bestBuyDate, bestSellDate)
```

1.

i	j loop
1	$2 \to n$ — $n-1$
2	$3 \to n$ — $n-2$
3	$4 \to n$ — $n-3$
⋮	⋮
k	$k+1 \to n$ — $n-k$

Terminate when $n-k=0$

$k=n$

$=(n-1)+(n-2)+(n-3)+\cdots+0$

$= \frac{n(n-1)}{2}$

$= \frac{n^2-n}{2}$

So, $O(n^2)$

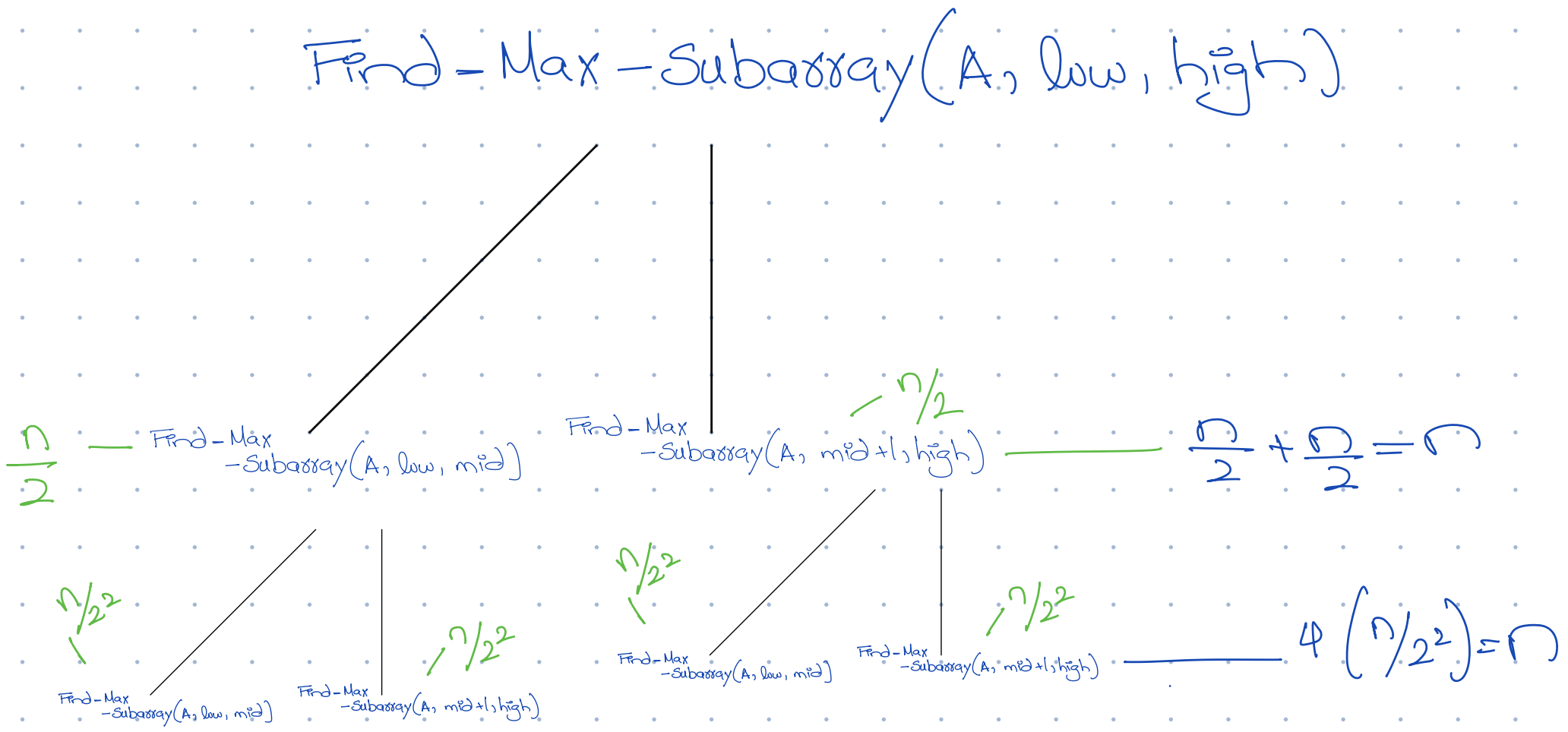
NOTE:
This is iterations of inner loop for each iteration of outer loop.
We don't need to multiply as this sum accounts for both loops

Algorithm:

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
 1  if high == low
 2       return (low, high, A[low])          // base case: only one element
 3  else mid = ⌊(low + high)/2⌋
 4       (left-low, left-high, left-sum) =
            FIND-MAXIMUM-SUBARRAY(A, low, mid)
 5       (right-low, right-high, right-sum) =
            FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
 6       (cross-low, cross-high, cross-sum) =
            FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 7       if left-sum ≥ right-sum and left-sum ≥ cross-sum
 8            return (left-low, left-high, left-sum)
 9       elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
10            return (right-low, right-high, right-sum)
11       else return (cross-low, cross-high, cross-sum)
```

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

2. Recursive Tree Method



Terminate when

$$\frac{n}{2^k} = 1$$

$$k = \log_2 n$$

So,

$$n \log_2(n)$$

$$= n + n + n + \dots$$

$$= kn$$

Substitution:

$$T(n) = 2T(n/2) + n \quad \text{— (i)}$$

put $n = \frac{n}{2}$ in (i)

$$T(n/2) = 2T(n/2^2) + n$$

put $T(n/2)$ in (i)

$$T(n) = 2^2 T(n/2^2) + n + n \quad \text{— (ii)}$$

put $n = \frac{n}{2^2}$ in (i)

$$T(n/2^2) = 2T(n/2^3) + n$$

put $T(n/2^2)$ in (ii)

$$T(n) = 2^3 T(n/2^3) + n + n + n$$

⋮ k^{th} iteration

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

Terminate when

$$\frac{n}{2^k} = 1$$

$$k = \log n$$

$$T(n) = 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n \times n$$

$$= nT(1) + n\log_2 n$$

$$= n + \frac{n}{2}\log_2 n$$

$$O(n \log n)$$

Problem 02

```
long long fastPower(long long a, long long n)
{
    if (n == 0)
        return 1;

    long long half = fastPower(a, n / 2);

    if (n % 2 == 0)
        return half * half;
    else
        return a * half * half;
}
```

T(n)

1

T(n/2)

1

1

1

1. $T(n) = T(n/2) + 1$

2. $T(n) = T(n/2) + 1$ — (i)

put $n = n/2$ in (i)

$T(n/2) = T(n/2^2) + 1$

put $T(n/2)$ in (i)

$T(n) = T(n/2^2) + 2$ — (ii)

put $n = n/2^2$ in (i)

$T(n/2^2) = T(n/2^3) + 1$

put $T(n/2^2)$ in (ii)

$T(n) = T(n/2^3) + 3$

⋮ kth

$T(n) = T\left(\frac{n}{2^k}\right) + k$

Terminate when

$\frac{n}{2^k} = 1$

This D asks for recurrence relation for worst case. How would best case look like? would it depend on the algo??

$$k = \log_2 n$$

$$T(n) = T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n$$

$$= T(1) + \log_2 n$$

$$= \log_2 n + 1$$

$$O(\log_2 n)$$